Assignment Code: DA-AG-012

Decision Tree | **Assignment**

Instructions: Carefully read each question. Use Google Docs, Microsoft Word, or a similar tool to create a document where you type out each question along with its answer. Save the document as a PDF, and then upload it to the LMS. Please do not zip or archive the files before uploading them. Each question carries 20 marks.

Total Marks: 100

Question 1: What is a Decision Tree, and how does it work in the context of classification?

Answer:

A Decision Tree is a supervised machine learning algorithm used for both classification and regression tasks. In classification, it predicts a category (class label) by learning decision rules from data features.

It Works in Classification

1 Start at the Root

The algorithm selects the best feature to split the dataset into groups.

Example:

> **"Is Age > 30?"**

This creates branches:

- **Yes**
- **No**

2 Choose the Best Split

The tree chooses splits using metrics like:

- **Gini Impurity**

- **Entropy (Information Gain)**

These measure how "pure" a group is:

- **A pure node contains mostly one class.**
- **The goal is to reduce impurity after each split.**

---

3 Recursive Splitting

Each subset is split again based on another feature:

Example:

- **Is Income > 50K?**
- **Has Credit Score > 700?**

This continues until:

- **Nodes are pure, or**
- **A stopping condition is met (max depth, min samples, etc.)**

---

4 Make a Prediction

To classify a new data point:

- **Start at the root.**
- **Follow the decision rules down the tree.**

Question 2: Explain the concepts of Gini Impurity and Entropy as impurity measures. How do they impact the splits in a Decision Tree?

Answer:

In a Decision Tree (classification), the goal at each split is to make the resulting groups as "pure" as possible — meaning each group contains mostly one class.

Two common impurity measures used to decide the best split are:

- **Gini Impurity**
- **Entropy (Information Gain)**

Both measure how mixed the classes are in a node.

---

1 Gini Impurity

🔹 Definition

Gini Impurity measures the probability of incorrectly classifying a randomly chosen sample if we randomly assign a label according to the class distribution in the node.

Formula

For a node with KKK classes:

Gini=1−∑i=1Kpi2

Where:

- **pi = proportion of class i in the node**

Entropy

🔹 Definition

Entropy measures the amount of uncertainty (or disorder) in a node.

It comes from information theory.

Formula

Entropy=−∑i=1Kpilog2(pi)

Question 3: What is the difference between Pre-Pruning and Post-Pruning in Decision Trees? Give one practical advantage of using each.

Answer:

1 Pre-Pruning (Early Stopping)

Pre-pruning stops the tree from growing before it becomes too complex.

The algorithm halts splitting when certain conditions are met.

2 Post-Pruning (Prune After Full Growth)

Post-pruning first grows the full tree (often until pure leaves), then removes unnecessary branches afterward.

Aspect	Pre-Pruning	Post-Pruning
When applied	During tree growth	After full tree is built
Risk	Underfitting	Slightly more computation
Speed	Faster	Slower
Accuracy	May miss patterns	Often better generalization

Question 4: What is Information Gain in Decision Trees, and why is it important for choosing the best split?

Answer:

Information Gain :

Information Gain measures how much a split reduces entropy (uncertainty) in a Decision Tree.

Formula:

IG=Entropy(parent)−Weighted Entropy(children)

it's important because:

- **It helps choose the best feature to split on.**
- **The split with the highest Information Gain is selected.**
- **Higher gain → purer child nodes → better classification.**

It tells us how much "information" we gain after splitting the data.

Question 5: What are some common real-world applications of Decision Trees, and what are their main advantages and limitations?

Answer:

Common Real-World Applications of Decision Trees

1 Finance

- **Credit scoring (loan approval)**
- **Fraud detection**
- **Risk assessment**

2 Healthcare

- **Disease diagnosis**
- **Treatment recommendation**
- **Patient risk prediction**

3 Marketing

- **Customer segmentation**
- **Churn prediction**
- **Targeted advertising**

4 Business & Operations

- **Sales forecasting**
- **Demand prediction**
- **Decision support systems**

5 Technology

- **Spam detection**
- **Recommendation systems (as part of ensemble models)**

Dataset Info:

- **Iris Dataset** for classification tasks (`sklearn.datasets.load_iris()` or provided CSV).
- **Boston Housing Dataset** for regression tasks (`sklearn.datasets.load_boston()` or provided CSV).

Question 6: Write a Python program to:

- Load the Iris Dataset
- Train a Decision Tree Classifier using the Gini criterion

- Print the model's accuracy and feature importances

(*Include your Python code and output in the code box below.*)

Answer:

```
# Import required libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# 1. Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# 2. Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# 3. Train Decision Tree Classifier using Gini criterion
clf = DecisionTreeClassifier(criterion='gini', random_state=42)
clf.fit(X_train, y_train)

# 4. Make predictions
y_pred = clf.predict(X_test)

# 5. Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)

# 6. Print results
print("Model Accuracy:", accuracy)
print("\nFeature Importances:")
for feature, importance in zip(iris.feature_names, clf.feature_importances_):
    print(f"{feature}: {importance:.4f}")
```

output:

```
Model Accuracy: 1.0

Feature Importances:
sepal length (cm): 0.0000
sepal width (cm): 0.0191
petal length (cm): 0.8933
petal width (cm): 0.0876
```

[redacted]

Question 7: Write a Python program to:

- Load the Iris Dataset
- Train a Decision Tree Classifier with `max_depth=3` and compare its accuracy to a fully-grown tree.

(*Include your Python code and output in the code box below.*)

Answer: # Import required libraries

```
from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score


# 1. Load the Iris dataset

iris = load_iris()

X = iris.data

y = iris.target


# 2. Split dataset into training and testing sets

X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.3, random_state=42

)


# 3. Fully-grown Decision Tree

full_tree = DecisionTreeClassifier(random_state=42)
```

```
full_tree.fit(X_train, y_train)

y_pred_full = full_tree.predict(X_test)

accuracy_full = accuracy_score(y_test, y_pred_full)


# 4. Decision Tree with max_depth=3

limited_tree = DecisionTreeClassifier(max_depth=3, random_state=42)

limited_tree.fit(X_train, y_train)

y_pred_limited = limited_tree.predict(X_test)

accuracy_limited = accuracy_score(y_test, y_pred_limited)


# 5. Print results

print("Accuracy of Fully-Grown Tree:", accuracy_full)

print("Accuracy of Tree with max_depth=3:", accuracy_limited)
```

Output:

```
Accuracy of Fully-Grown Tree: 1.0

Accuracy of Tree with max_depth=3: 1.0
```

Question 8: Write a Python program to:

- Load the Boston Housing Dataset
- Train a Decision Tree Regressor
- Print the Mean Squared Error (MSE) and feature importances

(*Include your Python code and output in the code box below.*)

Answer:

```
# Import required libraries
import pandas as pd
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score

# 1. Load Boston Housing dataset from OpenML
boston = fetch_openml(name="boston", version=1, as_frame=True)

X = boston.data
y = boston.target.astype(float)  # convert target to numeric

# 2. Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# 3. Train Decision Tree Regressor
regressor = DecisionTreeRegressor(random_state=42)
regressor.fit(X_train, y_train)

# 4. Predictions
y_pred = regressor.predict(X_test)

# 5. Evaluation
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error:", mse)
print("R2 Score:", r2)
```

Output:

```
Mean Squared Error: 11.588026315789474
R2 Score: 0.8444833592340152
```

[redacted]

Question 9: Write a Python program to:

- Load the Iris Dataset
- Tune the Decision Tree's `max_depth` and `min_samples_split` using GridSearchCV
- Print the best parameters and the resulting model accuracy

(*Include your Python code and output in the code box below.*)

Answer:

```
# Import required libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# 1. Load the Iris dataset
iris = load_iris()
X = iris.data
y = iris.target

# 2. Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# 3. Define the Decision Tree and parameter grid
dtree = DecisionTreeClassifier(random_state=42)
param_grid = {
    'max_depth': [2, 3, 4, 5, None],
    'min_samples_split': [2, 3, 4, 5]
}

# 4. Perform GridSearchCV
grid_search = GridSearchCV(dtree, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_train, y_train)
```

```
# 5. Best parameters and model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_

# 6. Predict on test set
y_pred = best_model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

# 7. Print results
print("Best Parameters:", best_params)
print("Test Set Accuracy:", accuracy)
```

Output:

```
Best Parameters: {'max_depth': 3, 'min_samples_split': 2}
Test Set Accuracy: 1.0
```


Question 10: Imagine you're working as a data scientist for a healthcare company that wants to predict whether a patient has a certain disease. You have a large dataset with mixed data types and some missing values.
Explain the step-by-step process you would follow to:

- Handle the missing values
- Encode the categorical features
- Train a Decision Tree model
- Tune its hyperparameters
- Evaluate its performance
  And describe what business value this model could provide in the real-world setting.

Answer:

1 Handle Missing Values

Step-by-step:

- **Identify missing data**
  - **Use `df.isnull().sum()` or `df.info()` to find missing entries.**

- **Impute missing values:**
    - **Numerical features: Replace missing values with mean or median (median is better if data is skewed).**
    - **Categorical features: Replace missing values with the mode or a special category like `"Unknown"`.**
    - **Alternatively, use KNN imputation or IterativeImputer for more advanced handling.**
- **Business Reasoning:**
    - **Proper handling ensures the model doesn't discard valuable patient records and reduces bias from missing data.**

2Encode Categorical Features

Decision Trees in scikit-learn require numerical input.

Approach:

- **Label Encoding:**
    - **For ordinal features (e.g., `"mild" < "moderate" < "severe"`), map categories to integers.**
- **One-Hot Encoding:**
    - **For nominal features (e.g., `"blood_type"`, `"gender"`), use `pd.get_dummies()` or `OneHotEncoder` to create binary columns.**
- **Business Reasoning:**
    - **Proper encoding ensures the tree can use all patient attributes effectively**

teps:

1. **Split data: `train_test_split(X, y, test_size=0.3, random_state=42)`**
2. **Initialize the model:**

from sklearn.tree import DecisionTreeClassifier

dtree = DecisionTreeClassifier(criterion='gini', random_state=42)

3. Train on the training set: `dtree.fit(X_train, y_train)`

4. Make predictions: `y_pred = dtree.predict(X_test)`

- Business Reasoning:
- Decision Trees handle mixed data types and missing values well (after preprocessing), and provide interpretable if–then rules for clinical decisions.

---

4 Tune Hyperparameters

Key hyperparameters to control complexity and avoid overfitting:

Hyperparameter	Purpose
`max_depth`	Limit tree depth to prevent overfitting
`min_samples_split`	Minimum samples to split a node
`min_samples_leaf`	Minimum samples per leaf to prevent tiny nodes
`max_features`	Limit features considered per split for robustness

GridSearchCV Example:

```
from sklearn.model_selection import GridSearchCV


param_grid = {

   'max_depth': [3, 5, 7, None],

   'min_samples_split': [2, 5, 10],
```

```
    'min_samples_leaf': [1, 2, 4]
}

grid = GridSearchCV(dtree, param_grid, cv=5, scoring='accuracy')
grid.fit(X_train, y_train)

best_model = grid.best_estimator_
```

- **Business Reasoning:**
    - **Hyperparameter tuning balances model accuracy and generalizability, avoiding overfitting to the training data.**

---

5 Evaluate Performance

Metrics for classification:

- **Accuracy: Percentage of correct predictions**
- **Precision: How many predicted positives are true positives (important for avoiding false alarms)**
- **Recall (Sensitivity): How many actual positives were detected (critical in healthcare)**
- **F1-score: Balance of precision and recall**
- **ROC-AUC: Overall ability to distinguish diseased vs. healthy patients**

```
from sklearn.metrics import classification_report, roc_auc_score

print(classification_report(y_test, y_pred))
print("ROC-AUC:", roc_auc_score(y_test, dtree.predict_proba(X_test)[:,1]))
```

6 Business Value

A well-trained Decision Tree model in this context provides:

- **Early detection of diseases → faster treatment → improved patient outcomes**
- **Risk stratification → helps prioritize high-risk patients**
- **Decision support for doctors → interpretable rules explain why a patient is predicted positive**
- **Cost efficiency → reduces unnecessary tests and hospital visits**